تفاوت معماری Monolithو — Microservicesبه زبان ساده

۲۹ مهر ۱۴۰۵

فهرست مطالب

۱ معماری یعنی چه؟ (خیلی کوتاه) ۱

۲ Monolith: همه‌چیز در یک جعبه ۱

۱.۲ مزایا ۱

۲.۲ معایب ۱

۳ Microservices: تیمی از سرویس‌های کوچک ۳

۱.۳ مزایا ۳

۲.۳ معایب ۴

۴ مقایسه مستقیم ۴

۵ کِی از کدوم استفاده کنیم؟ ۴

۱.۵ Monolith انتخاب کن اگر: ۵

۲.۵ Microservices انتخاب کن اگر: ۵

۱ معماری یعنی چه؟ (خیلی کوتاه)

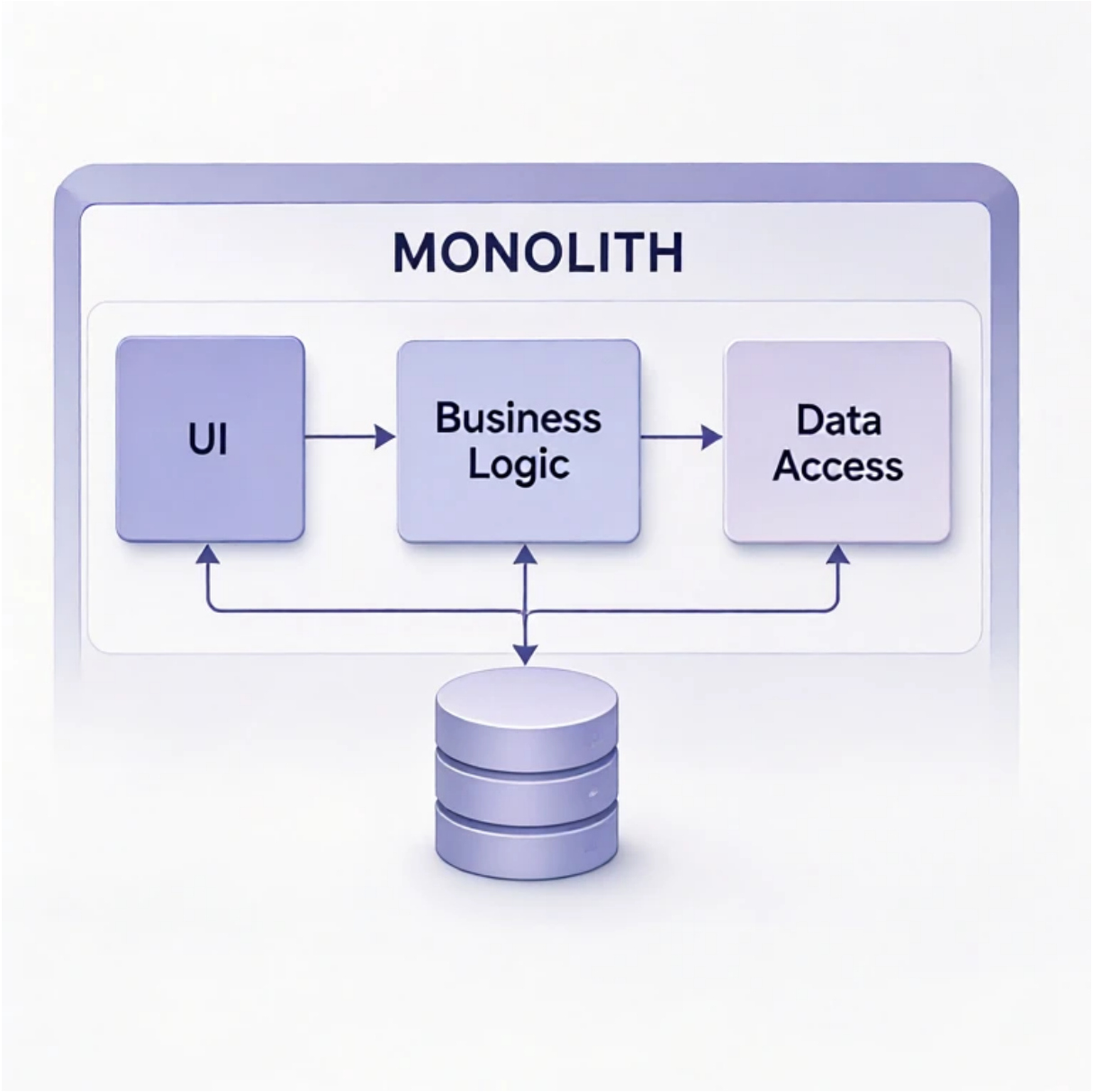
معماری نرم‌افزار یعنی نقشه‌ی کلی پروژه: کدها چطور تقسیم می‌شوند، به هم وصل می‌شوند و کنار هم اجرا می‌شوند. دو مسیر اصلی جلوی ماست: همه‌ی کد در یک برنامه‌ی واحد (Monolith) یا شکستن آن به سرویس‌های کوچک مستقل (Microservices). این سند با زبان ساده نشان می‌دهد هرکدام چیست و کجا به کار می‌آید.

۲ Monolith: همه‌چیز در یک جعبه

در معماری یکپارچه، همه‌ی بخش‌های برنامه – رابط کاربری، منطق کسب‌وکار و دسترسی به دیتابیس – داخل **یک پروژه و یک فرآیند اجرایی** زندگی می‌کنند. مثل یک جعبه‌ابزار کامل: همه چیز یک‌جا است و برای برداشتن هر ابزاری فقط کافی است درِ همان جعبه را باز کنی.

۱.۲ مزایا

- **سادگی در شروع:** یک پروژه، یک دیپلوی، یک دیتابیس. راه‌اندازی‌اش سریع است.



شکل ۱: نمودار معماری Monolith

- **اشکال‌زدایی راحت‌تر:** همه‌چیز در یک فرآیند اجرا می‌شود؛ با یک اجرای ساده (Debug) کل برنامه را می‌بینی.
- **سرعت فراخوانی‌ها:** بخش‌های داخلی با فراخوانی تابع (in-process) صحبت می‌کنند، نه تماس شبکه‌ای — پس سریع و قابل‌اعتماد است.
- **تراکنش‌های یکپارچه:** چند عملیات را می‌توان در یک تراکنش دیتابیسی با هم ثبت کرد.

۲.۲ معایب

- با بزرگ شدن پروژه، کد به راحتی به یک «توپ گِلی بزرگ» (Big Ball of Mud) تبدیل می‌شود؛ مرز ماژول‌ها راحت دور زده می‌شود و بی‌نظمی اسباب‌کشی می‌کند.
- برای تغییر کوچک باید کل برنامه را دوباره استقرار داد؛ حتی یک خط تغییر یعنی دیپلوی کامل.
- مقیاس‌پذیری درشت‌دانه است: اگر فقط یک بخش شلوغ شود، باید کل برنامه را مقیاس دهی کرد.
- مجبوری با یک فناوری واحد بمانی؛ نمی‌شود هر بخش را با زبان یا ابزار دلخواه ساخت.

نکته: «Monolith» دشنام نیست؛ کاملاً ممکن است یک سیستم یکپارچه، تمیز و ماژولار ساخته شود — فقط به نظم و انضباط تیم بیشتر نیاز دارد.

۳ Microservices: تیمی از سرویس‌های کوچک

در معماری ریزسرویس، برنامه به **سرویس‌های کوچک و مستقل** شکسته می‌شود؛ هر سرویس یک کار مشخص انجام می‌دهد (مثلاً کاربران، سفارش‌ها، پرداخت)، دیتابیس خودش را دارد و با بقیه از طریق **API** حرف می‌زند. مثل یک تیم متخصص: هر نفر کار خودش را بلد است و برای هماهنگی، پیام می‌فرستد.

۱.۳ مزایا

- **مرزهای ماژولی محکم:** چون هر سرویس جدا است، دور زدن مرزها کار سختی است؛ ساختار تمیز بیشتر حفظ می‌شود و هر تیم مسئولیت روشنی دارد.
- **استقرار مستقل:** هر سرویس جداگانه دیپلوی می‌شود؛ خرابی یک سرویس به‌راحتی کل سیستم را نمی‌اندازد.
- **آزادی فناوری:** می‌شود هر سرویس را با زبان، فریم‌ورک و دیتابیس مناسب خودش ساخت.
- **مقیاس‌دهی دقیق:** فقط همان سرویسی که زیر فشار است، چند نمونه می‌شود؛ نه کل برنامه.
- مناسب تیم‌های بزرگ: تیم‌های مختلف روی سرویس‌های جدا کار می‌کنند و همدیگر را گیر نمی‌اندازند.

۲.۳ معایب

- **پیچیدگی سیستم توزیع‌شده:** تماس‌های شبکه‌ای کندتر از فراخوانی تابع هستند و ممکن است هر لحظه شکست بخورند؛ باید برای خرابی طراحی کرد.
- **سازگاری نهایی :(Eventual Consistency)** چون هر سرویس دیتابیس خودش را دارد، داده‌ها ممکن است لحظه‌ای ناهماهنگ باشند و تراکنش‌های توزیع‌شده سخت می‌شوند.
- **پیچیدگی عملیاتی:** مدیریت ده‌ها سرویس، مانیتورینگ، لاگ مرکزی و استقرار پیوسته، به تیم عملیات با تجربه نیاز دارد.
- **دیباگ سخت‌تر:** ردیابی یک درخواست که بین چند سرویس می‌چرخد، به‌مراتب سخت‌تر از یک برنامه‌ی واحد است.

فاولر این هزینه‌ی اضافه را «Microservice Premium» می‌نامد: اگر سیستم به‌قدر کافی پیچیده نباشد، این هزینه ارزشش را ندارد.



شکل ۲: نمودار معماری Microservices

۴ مقایسه مستقیم

خلاصه‌ی تفاوت‌های کلیدی در یک نگاه:

معیار	Monolith	Microservices
ساختار کد	یک پروژه و یک فرآیند واحد	سرویس‌های کوچک مستقل با API
دیتابیس	معمولاً یک دیتابیس مشترک	هر سرویس دیتابیس خودش
استقرار	دیپلوی کامل با هر تغییر	هر سرویس جداگانه دیپلوی می‌شود
مقیاس‌پذیری	کل برنامه با هم	فقط سرویس شلوغ مقیاس می‌گیرد
فناوری	یک استک واحد	تنوع زبان و ابزار آزاد است
سرعت فراخوانی‌ها	سریع (فراخوانی تابع داخلی)	کندتر (تماس شبکه‌ای) و در معرض خطا
سازگاری داده	تراکنش یکپارچه آسان	سازگاری نهایی و تراکنش توزیع‌شده سخت
اشکال‌زدایی	ساده با دیباگ معمولی	نیاز به ردیابی توزیع‌شده و لاگ مرکزی
تیم مناسب	تیم کوچک تا متوسط	تیم‌های بزرگ و چندتیمی
هزینه عملیاتی	کم	بالا (زیرساخت و تیم عملیات حرفه‌ای)

در یک جمله: Monolith **سرعت شروع و سادگی** می‌دهد و Microservices **استقلال و مقیاس** — به بهای پیچیدگی توزیع‌شده.

۵ کِی از کدوم استفاده کنیم؟

پاسخ کوتاه: **با Monolith شروع کن، مگر اینکه دلیل قوی برای ریزسرویس داشته باشی.** فاولر هم استراتژی «اول یکپارچه» (Monolith First) را توصیه می‌کند، چون مرزهای درست سرویس‌ها را معمولاً فقط بعد از فهم دامنه‌ی مسئله می‌توان درست تعیین کرد.

۱.۵ Monolith انتخاب کن اگر:

- پروژه یا محصول جدید است و دامنه‌ی مسئله هنوز کاملاً روشن نیست.
- تیم کوچک است (چند نفر تا حدود ده نفر).
- ترافیک پیش‌بینی‌پذیر است و مقیاس‌دهی درشت‌دانه کافی است.

۲.۵ Microservices انتخاب کن اگر:

- سیستم واقعاً بزرگ و پیچیده است و تیم‌های متعدد باید موازی کار کنند.
- بخش‌های مختلف سیستم نیازهای مقیاس یا فناوری متفاوت دارند.
- زیرساخت و تیم عملیاتِ خبره (استقرار خودکار، مانیتورینگ، لاگ مرکزی) را داری.

مسیر عملی و رایج در صنعت: با یک Monolith تمیز و ماژولار شروع کن؛ اگر روزی بخش خاصی از سیستم به مقیاس یا استقلال بیشتری نیاز پیدا کرد، همان بخش را به یک سرویس مستقل تبدیل کن.